

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0101**

Assessment Tool Weightage: 40%

No. of Questions: 1

I understand that any violation of academic integrity rules will result in disciplinary action.

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	

Total	100%	
--------------	-------------	--

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil

	conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

		statements.			
6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

Automobile Fault Diagnosis Using Different Knowledge Representation Models

1. Problem Statement and Objectives

Problem Statement

An automobile service center needs an intelligent system to identify possible vehicle faults based on symptoms observed in a vehicle. Common symptoms include:

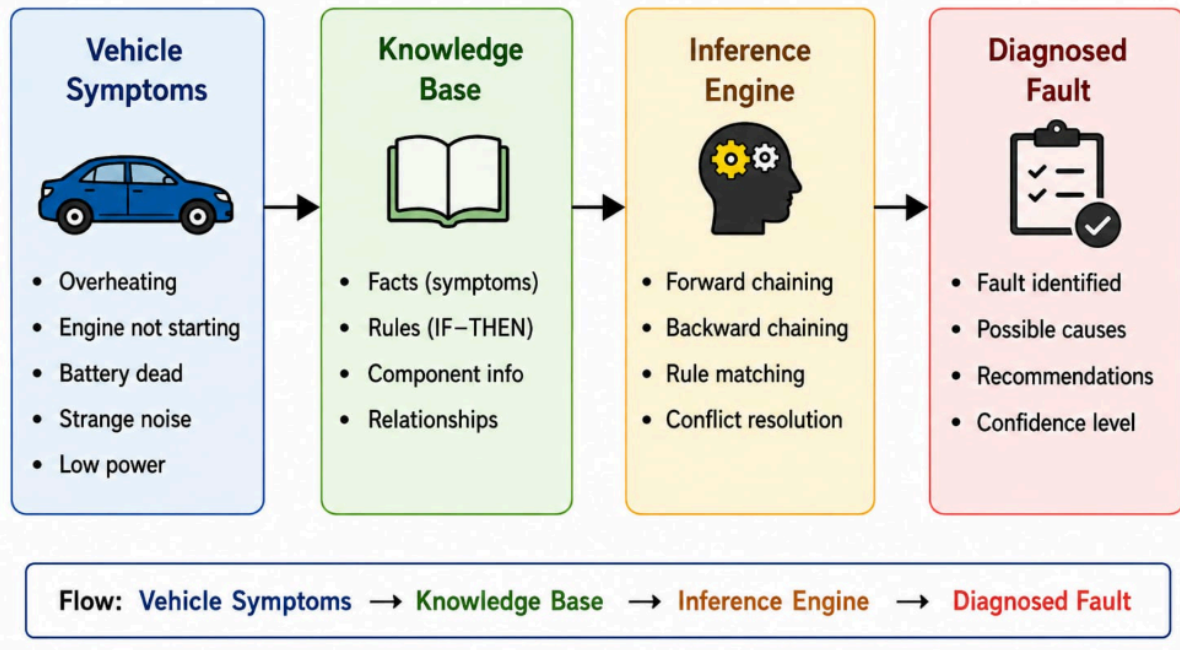
- Engine overheating
- Starting failure
- Abnormal engine noise
- Low mileage
- Warning indicators

The system should use these symptoms to identify possible faults such as:

- Cooling system failure
- Weak or dead battery
- Low engine oil
- Fuel system problem
- Engine problem

The automobile fault diagnosis problem can be represented using **Production Rules, Propositional Logic, First-Order Logic (FOL), and Prolog**. The different models can then be compared based on expressiveness, inference mechanism, complexity, scalability, and suitability.

1. Automobile fault diagnosis overview diagram



Objectives

1. Identify important automobile symptoms and faults.
2. Represent automobile knowledge using production rules.
3. Model the problem using propositional logic.
4. Represent relationships using first-order logic.
5. Implement the knowledge using Prolog.
6. Demonstrate forward and backward chaining.
7. Compare all four modelling approaches.
8. Select the most suitable model for automobile fault diagnosis.

2. Domain Knowledge and Fact Identification

Important Entities

Entity	Examples
Vehicle	Car, bike, truck
Symptom	Overheating, starting failure, abnormal noise
Component	Battery, engine, radiator, oil system
Fault	Battery failure, cooling failure, low oil
Warning	Check-engine light, temperature warning

Symptoms

- Engine overheating
- Vehicle does not start
- Abnormal engine noise
- Low mileage
- Warning light is ON
- Low engine oil
- Weak battery
- Low coolant

Possible Faults

Symptoms	Possible Fault
Overheating + low coolant	Cooling system failure
Starting failure + weak battery	Battery failure
Abnormal noise + low oil	Low engine oil
Low mileage + fuel problem	Fuel system problem
Warning light + abnormal noise	Engine problem

Basic Facts

For example, consider a vehicle **car1**:

- car1 has overheating
- car1 has low coolant
- car1 has starting failure
- car1 has weak battery
- car1 has abnormal noise
- car1 has low engine oil

These facts are used by the inference system to determine the possible fault.

3. Production Rule Model

Production rules represent knowledge in the form:

IF condition THEN conclusion

Rules

Rule 1: Cooling System Failure

IF engine_overheating AND low_coolant
THEN cooling_system_failure.

Rule 2: Battery Failure

IF starting_failure AND weak_battery
THEN battery_failure.

Rule 3: Low Engine Oil

IF abnormal_noise AND low_engine_oil
THEN oil_problem.

Rule 4: Fuel System Problem

IF low_mileage AND fuel_problem
THEN fuel_system_fault.

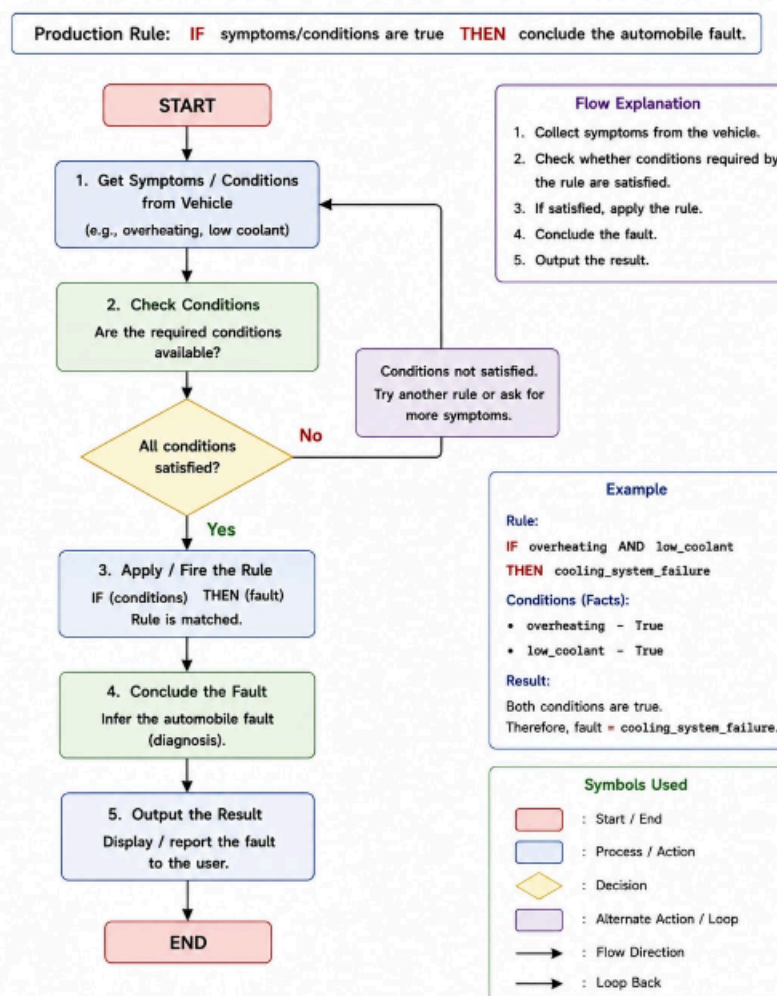
Rule 5: Engine Fault

IF warning_light AND abnormal_noise
THEN engine_fault.

Rule 6: Serious Engine Problem

IF engine_overheating AND abnormal_noise
THEN serious_engine_problem.

2. Production Rule Flowchart



Example

Given:

- engine_overheating
- low_coolant

The system applies Rule 1:

**IF engine_overheating AND low_coolant
THEN cooling_system_failure**

Therefore:

Conclusion: Cooling system failure

Production rules are easy to understand because the reasoning is expressed directly using IF–THEN statements.

4. Propositional Logic Model

In propositional logic, each statement is represented using a proposition that is either **True** or **False**.

Propositions

Let:

- O = Engine is overheating
- C = Coolant level is low
- S = Vehicle has starting failure
- B = Battery is weak
- N = Engine produces abnormal noise
- L = Engine oil is low
- M = Vehicle has low mileage
- F = Fuel system has a problem
- W = Warning light is ON
- D = Cooling system failure
- E = Battery failure
- P = Oil problem
- T = Fuel system fault
- G = Engine fault

Logical Rules:

Rule 1:

$$O \wedge C \rightarrow D$$

Meaning:

If the engine is overheating AND coolant is low, then there is a cooling system failure.

Rule 2:

$$S \wedge B \rightarrow E$$

Rule 3:

$$N \wedge L \rightarrow P$$

Rule 4:

$$M \wedge F \rightarrow T$$

Rule 5:

$$W \wedge N \rightarrow G$$

3. Propositional Logic Diagram

Show: **Overheating $\wedge$ Low Coolant $\rightarrow$ Cooling System Failure**



Propositional Expression : $O \wedge L \rightarrow C$

Meaning of Propositions

O : The vehicle is overheating.
L : The coolant level is low.
C : The cooling system has failed.

Conclusion

The statement $(O \wedge L \rightarrow C)$ is false only when both Overheating and Low Coolant are true, but Cooling System Failure is false.

Truth Table

O	L	C	$O \wedge L$	$O \wedge L \rightarrow C$
F	F	F	F	T
F	F	T	F	T
F	T	F	F	T
F	T	T	F	T
T	F	F	F	T
T	F	T	F	T
T	T	F	T	F
T	T	T	T	T

Example Inference

Given:

O = True

C = True

Therefore:

$O \wedge C = \text{True}$

Hence:

D = True

So, the system concludes:

Cooling system failure

Limitation

Propositional logic is simple, but it cannot easily represent different vehicles, components, or relationships between objects because each statement is treated as a complete proposition.

5. First-Order Logic Model

First-Order Logic is more expressive because it can represent **objects, properties, relationships, variables, and quantifiers**.

Predicates:

Let:

- $\text{overheating}(x)$ – vehicle x is overheating
- $\text{low_coolant}(x)$ – vehicle x has low coolant
- $\text{starting_failure}(x)$ – vehicle x has starting failure
- $\text{weak_battery}(x)$ – vehicle x has a weak battery
- $\text{abnormal_noise}(x)$ – vehicle x has abnormal noise
- $\text{low_oil}(x)$ – vehicle x has low engine oil
- $\text{cooling_failure}(x)$ – vehicle x has cooling system failure
- $\text{battery_failure}(x)$ – vehicle x has battery failure
- $\text{oil_problem}(x)$ – vehicle x has an oil problem
- $\text{engine_fault}(x)$ – vehicle x has an engine fault

FOL Rules**Rule 1**

$\forall x [(\text{overheating}(x) \wedge \text{low_coolant}(x)) \rightarrow \text{cooling_failure}(x)]$

Meaning:

For every vehicle x , if x is overheating and has low coolant, then x has a cooling system failure.

Rule 2

$\forall x [(\text{starting_failure}(x) \wedge \text{weak_battery}(x)) \rightarrow \text{battery_failure}(x)]$

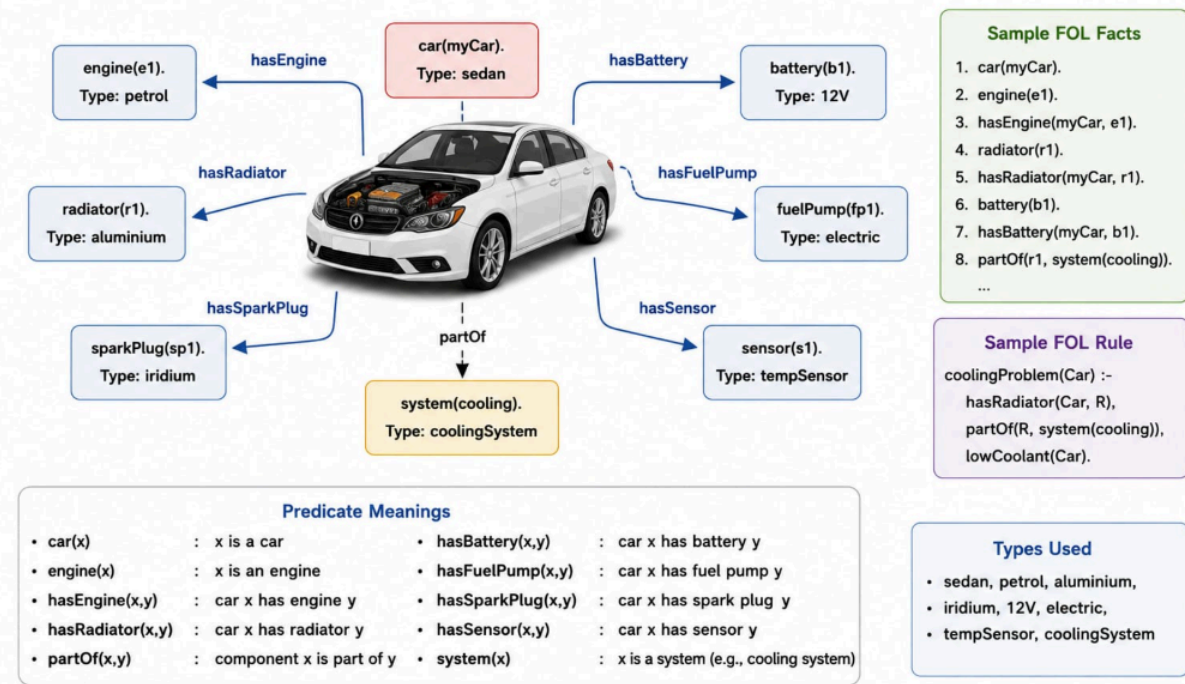
Rule 3

$\forall x [(\text{abnormal_noise}(x) \wedge \text{low_oil}(x)) \rightarrow \text{oil_problem}(x)]$

Rule 4

$\forall x [(\text{warning_light}(x) \wedge \text{abnormal_noise}(x)) \rightarrow \text{engine_fault}(x)]$

4. First-Order Logic diagram (Car and its components/relationships)



Facts

overheating(car1)
 low_coolant(car1)
 starting_failure(car1)
 weak_battery(car1)
 abnormal_noise(car1)
 low_oil(car1)

Inference

From:

overheating(car1)

and

low_coolant(car1)

using:

$\forall x [(overheating(x) \wedge low_coolant(x)) \rightarrow cooling_failure(x)]$

we derive:

cooling_failure(car1)

Similarly:

starting_failure(car1) + weak_battery(car1)

$\rightarrow$ **battery_failure(car1)**

Advantage of FOL

FOL can represent relationships between many vehicles and their components. Therefore, it is more expressive than simple propositional logic.

6. Prolog Model and Implementation

Prolog represents knowledge using **facts and rules** and automatically performs logical inference.

Prolog Code:

```
% Automobile Fault Diagnosis

% Facts
overheating(car1).
low_coolant(car1).
starting_failure(car1).
weak_battery(car1).
abnormal_noise(car1).
low_oil(car1).
low_mileage(car1).
fuel_problem(car1).
warning_light(car1).

% Rules
fault(Vehicle, cooling_system_failure) :-
    overheating(Vehicle),
    low_coolant(Vehicle).

fault(Vehicle, battery_failure) :-
    starting_failure(Vehicle),
    weak_battery(Vehicle).

fault(Vehicle, oil_problem) :-
    abnormal_noise(Vehicle),
    low_oil(Vehicle).

fault(Vehicle, fuel_system_fault) :-
    low_mileage(Vehicle),
    fuel_problem(Vehicle).

fault(Vehicle, engine_fault) :-
    warning_light(Vehicle),
    abnormal_noise(Vehicle).

fault(Vehicle, serious_engine_problem) :-
```

```
overheating(Vehicle),  
abnormal_noise(Vehicle).
```

Prolog Queries and Outputs

Query 1

?- fault(car1, Fault).

Output

```
Fault = cooling_system_failure ;  
Fault = battery_failure ;  
Fault = oil_problem ;  
Fault = fuel_system_fault ;  
Fault = engine_fault ;  
Fault = serious_engine_problem.
```

Query 2

?- fault(car1, battery_failure).

Output

true.

This means that the system successfully identifies a **battery failure** based on the available symptoms.

Query 3

?- fault(car1, cooling_system_failure).

Output

true.

Therefore, Prolog can automatically derive faults from the stored facts and rules.

7. Forward and Backward Chaining Demonstration

A. Forward Chaining

Forward chaining is **data-driven reasoning**.

It starts with known facts and repeatedly applies rules to derive new conclusions.

Given Facts

1. overheating(car1)
2. low_coolant(car1)
3. starting_failure(car1)
4. weak_battery(car1)

Step 1

Known:

```
overheating(car1)  
low_coolant(car1)
```

Apply Rule:

IF overheating AND low coolant
 THEN cooling system failure.
 Therefore:
cooling_system_failure(car1)

Step 2

Known:

starting_failure(car1)

weak_battery(car1)

Apply Rule:

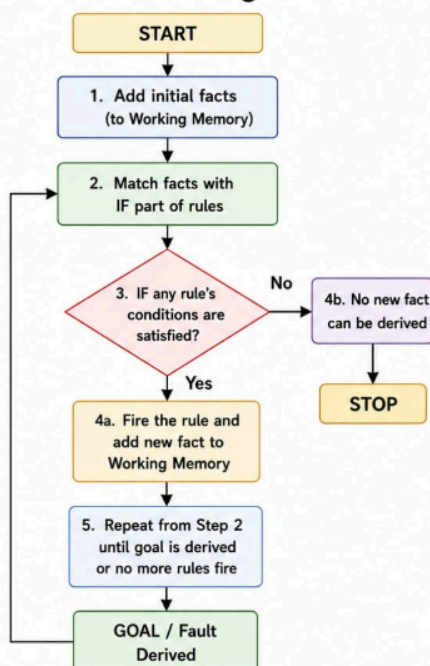
IF starting_failure AND weak_battery
 THEN battery_failure.

Therefore:

battery_failure(car1)

Forward Chaining Sequence

6. Forward Chaining Flowchart



Knowledge Base (Example)

Facts (Initial)	Rules
- overheating. - low_coolant.	R1: cooling_system_failure :- overheating, low_coolant. R2: coolant_leak :- low_coolant, radiator_leak. R3: radiator_leak :- old_coolant. R4: old_coolant :- recommend_flush.

Forward Chaining Inference Steps

Step	Working Memory (Known Facts)	Rule Fired	New Fact Added	Goal Reached?
0	{overheating, low_coolant}	—	—	No
1	{overheating, low_coolant}	R1	cooling_system_failure	Yes (if goal is R1)
2	{overheating, low_coolant, cooling_system_failure}	—	—	Yes
3	{overheating, low_coolant, cooling_system_failure}	—	—	No (continuing for more facts)
4	{overheating, low_coolant, cooling_system_failure}	—	—	No
5	{overheating, low_coolant, cooling_system_failure}	—	—	No
6	{overheating, low_coolant, cooling_system_failure}	—	—	No
7	{overheating, low_coolant, cooling_system_failure}	—	—	No

Explanation

Forward chaining starts with known facts and repeatedly applies rules whose conditions are satisfied to derive new facts, until the goal (fault) is obtained or no more rules can be fired.

Example Goal / Fault

cooling_system_failure.
 (Derived in Step 1)

B. Backward Chaining

Backward chaining is **goal-driven reasoning**.

It starts with a goal and works backward to determine whether the required facts are available.

Goal

Suppose the goal is:

battery_failure(car1)

The system searches for a rule that can conclude battery_failure.

It finds:

IF starting_failure AND weak_battery

THEN battery_failure.

Now it checks the conditions.

Step 1

Is:

starting_failure(car1)

true?

Yes.

Step 2

Is:

weak_battery(car1)

true?

Yes.

Step 3

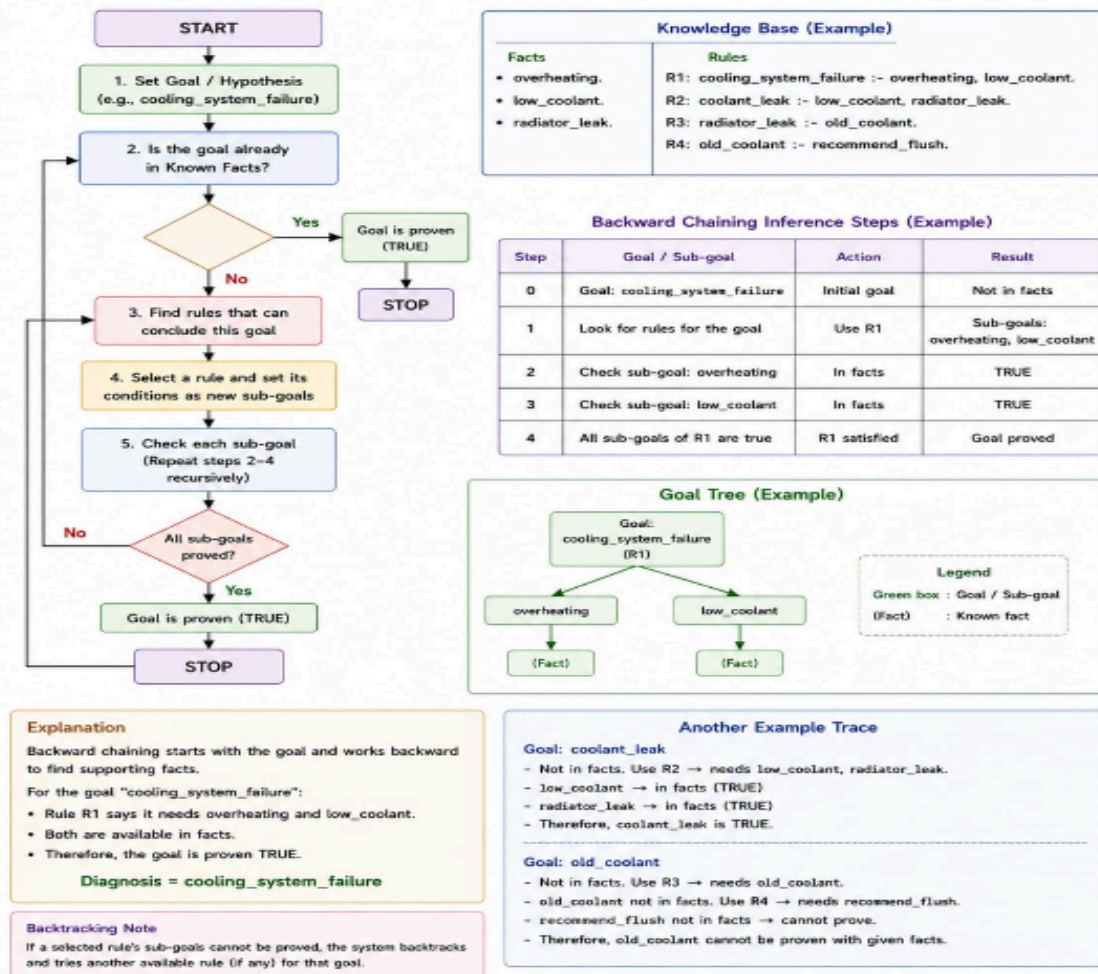
Both conditions are true.

Therefore:

battery_failure(car1) = TRUE

Backward Chaining Sequence

7. Backward Chaining Flowchart



Difference

Forward Chaining	Backward Chaining
Starts with facts	Starts with goal
Data-driven	Goal-driven
Derives possible conclusions	Checks whether a conclusion can be proved
Useful when many facts are available	Useful when a specific diagnosis is required

8.Comparative Analysis

The assignment requires comparison based on parameters such as expressiveness, inference, complexity, scalability, flexibility, explainability, and suitability.

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	IF–THEN rules	True/False propositions	Predicates, variables and relationships	Facts and rules
Expressiveness	Medium	Low	High	High
Rule Representation	Very simple	Logical implications	Predicate-based rules	Horn clauses
Inference Mechanism	Forward/Backward	Logical inference	Logical inference	Backward chaining
Forward Chaining	Easily supported	Possible through inference	Possible	Not the main mechanism
Backward Chaining	Easily supported	Possible	Supported conceptually	Naturally supported
Handling Relationships	Limited	Poor	Excellent	Excellent
Scalability	Medium	Low to Medium	High but complex	High for rule-based systems
Explainability	Very high	High	High	High
Ease of Implementation	Very easy	Easy	Moderate	Easy to Moderate
Suitability	Good for simple diagnosis	Good for simple facts	Excellent for complex domains	Excellent for practical diagnosis

Detailed Comparison

Production Rules

Production rules are easy to understand and directly represent diagnostic knowledge. They are highly explainable because each conclusion can be traced to an IF–THEN rule.

Propositional Logic

Propositional logic is simple and mathematically clear. However, it cannot naturally represent individual vehicles, components, or relationships.

First-Order Logic

FOL is highly expressive. It can represent vehicles, components, properties, and relationships using variables and predicates. However, the representation can become more complex.

Prolog

Prolog combines facts, rules, and automated logical inference. It is particularly suitable for expert systems because it can directly implement the knowledge base and answer diagnostic queries.

9. Results, Discussion and Recommended Model

Results

The four models successfully represent automobile fault diagnosis:

- Production rules clearly represent diagnostic rules.
- Propositional logic represents simple conditions and conclusions.
- First-Order Logic represents entities and relationships.
- Prolog provides an executable implementation of the knowledge base.

Recommended Model

Prolog is recommended as the most suitable model for automobile fault diagnosis.

Reasons

1. It directly supports facts and rules.
2. It provides automatic logical inference.
3. It supports backward chaining naturally.
4. It is easy to add new vehicle faults and symptoms.
5. Diagnostic queries can be executed directly.
6. The reasoning process is understandable.
7. It is suitable for implementing rule-based expert systems.

Production Rules vs Prolog

Production rules are excellent for representing knowledge, but Prolog is better for **actual implementation** because the rules can be executed automatically.

Limitations

The proposed system uses predefined symptoms and deterministic rules. Real automobile diagnosis may involve:

- Multiple simultaneous faults
- Uncertain sensor readings
- Missing information
- Different vehicle models
- Complex component relationships

Possible Improvements

The system can be improved by:

- Adding more vehicle components and faults.
- Including sensor data.
- Using probabilities or fuzzy logic for uncertain symptoms.

- Adding more diagnostic rules.
- Connecting the system with real-time vehicle data.
- Developing a user-friendly interface.

10. Conclusion, References and GitHub

Conclusion

Automobile fault diagnosis can be effectively represented using different knowledge representation techniques. Production rules provide simple and understandable IF–THEN knowledge. Propositional logic provides a basic mathematical representation of facts and conclusions. First-Order Logic provides greater expressiveness by representing objects, properties, variables, and relationships. Prolog provides an executable rule-based implementation with automated inference.

Among the four approaches, **Prolog is the most suitable for implementing an automobile fault diagnosis expert system**, while First-Order Logic is the most expressive for representing complex relationships. Production rules are the easiest for humans to understand, and propositional logic is suitable for simple problems.

Therefore, a practical automobile expert system can use **production-rule knowledge represented and implemented through Prolog**, with FOL concepts used when more complex relationships are required.

References

1. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*.
2. Ivan Bratko, *Prolog Programming for Artificial Intelligence*.
3. George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*.
4. Course material on Artificial Intelligence and Expert Systems.